

# Chapter 10: Production Resilience, Defensive Prompting, & Rate Limits

**The Non-Deterministic Nightmare** Writing an LLM agent script in a Jupyter Notebook that succeeds once is easy. Running an agent that executes 100,000 workflows a day across enterprise customers without silently failing, hallucinating corrupt JSON, or draining your credit card via runaway recursive loops is an entirely different engineering discipline.

Production Agentic AI engineering is **defensive engineering**. This chapter covers schema healing, token bucket rate limiters with jitter, multi-provider fallback cascades, and circuit breakers.

## 1. Schema Enforcement & Healing with Pydantic & Instructor

LLMs frequently generate invalid JSON: trailing commas, unescaped quotes, or markdown code fences (```) wrapped around payloads.

flowchart TD

```
Prompt["User Request"] → LLM["LLM Generates Raw Response"]
LLM → Parse{"Pydantic Validation Check"}
Parse -- Valid → Success["Return Strongly-Typed Object"]
Parse -- Invalid (e.g. Missing Field) → Refine["Build Schema Error Context"]
Refine → RepairPrompt["Reprompt LLM: 'Your response failed validation with: {error}'"]
RepairPrompt → LLM
```

## Self-Healing Extraction Pipeline

```

from pydantic import BaseModel, Field, ValidationError
from typing import List, Optional
import json

class DatabaseQueryPlan(BaseModel):
    target_table: str = Field(description="Name of the table to query")
    columns: List[str] = Field(description="Specific columns to project")
    where_clause: Optional[str] = Field(None, description="SQL WHERE filter clause")
    limit: int = Field(default=10, ge=1, le=100, description="Row limit between 1 and 100")

def execute_with_self_healing(llm_client, user_prompt: str, max_retries: int = 3) → Data:
    """
    Guarantees typed output by piping Pydantic validation errors back into the LLM
    context for iterative correction.
    """
    conversation = [
        {"role": "system", "content": f"You are a SQL planner. Output pure JSON adhering"},
        {"role": "user", "content": user_prompt}
    ]

    for attempt in range(max_retries):
        raw_response = llm_client.complete(conversation)

        # Clean markdown wrappers if present
        clean_text = raw_response.strip()
        if clean_text.startswith("```json"):
            clean_text = clean_text[7:]
        if clean_text.endswith("```"):
            clean_text = clean_text[:-3]

        try:
            parsed_data = json.loads(clean_text)
            validated_obj = DatabaseQueryPlan.model_validate(parsed_data)
            return validated_obj # 100% Guaranteed valid type!
        except (json.JSONDecodeError, ValidationError) as err:
            if attempt == max_retries - 1:
                raise RuntimeError(f"Agent failed to produce valid schema after {max_retr}

            # Feed the exact machine validation error back to the model!
            conversation.append({"role": "assistant", "content": raw_response})

```

```
conversation.append({
    "role": "user",
    "content": f"Schema validation failed: {str(err)}. Return ONLY the correct"
})
```

## 2. Rate Limits, Throttling, & Full Jitter Backoff

When you scale to 50 concurrent agents, OpenAI or Anthropic will immediately return `HTTP 429 Too Many Requests`.

### Why Exponential Backoff Without Jitter Causes Avalanches

If 50 agents hit a rate limit at the exact same millisecond and all sleep for  $2^1 = 2$  seconds, they will all wake up at the exact same millisecond and collide again!

$$\text{Sleep Time} = \text{random}(0, \min(M, B \times 2^{\text{attempt}}))$$

graph TD

Request["Agent API Call"] → Check{"HTTP 429 Rate Limit?"}

Check -- No → Success["Process Result"]

Check -- Yes → Jitter["Calculate Backoff with Full Jitter"]

Jitter → Sleep["Asynchronous Sleep (Non-blocking)"]

Sleep → Request

## Production Rate Limiter with Full Jitter

```
import asyncio
import random
import time

class ResilientLLMCaller:
    def __init__(self, base_delay: float = 0.5, max_delay: float = 30.0, max_retries: int = 5):
        self.base_delay = base_delay
        self.max_delay = max_delay
        self.max_retries = max_retries

    async def call_with_backoff(self, api_func, *args, **kwargs):
        for attempt in range(self.max_retries):
            try:
                return await api_func(*args, **kwargs)
            except Exception as e:
                # Check for rate limit or transient 5xx network errors
                is_rate_limit = "429" in str(e) or "RateLimitError" in type(e).__name__
                if not is_rate_limit or attempt == self.max_retries - 1:
                    raise e

                # Full Jitter formula: random float between 0 and 2^attempt * base
                ceiling = min(self.max_delay, self.base_delay * (2 ** attempt))
                sleep_seconds = random.uniform(0, ceiling)

                print(f"[RETRY {attempt + 1}/{self.max_retries}] 429 detected. Jitter backoff: {sleep_seconds}s")
                await asyncio.sleep(sleep_seconds)
```

## 3. Multi-Provider Fallback Cascade

Relying on a single LLM provider in production is an existential risk. When OpenAI has an API outage, your business stops. A resilient agent architecture uses a **Cascade**:

flowchart LR

```

Task["Agent Task"] → ProviderA["Primary: Claude 3.5 Sonnet"]
ProviderA -- "Success" → Complete["Return Task Result"]
ProviderA -- "Timeout / 500 Outage" → ProviderB["Secondary: GPT-4o"]
ProviderB -- "Success" → Complete
ProviderB -- "Outage" → ProviderC["Fallback: Local DeepSeek / Ollama"]
ProviderC → Complete

```

## Cascade Implementation

```

from typing import List, Callable, Any

class LLMProviderCascade:
    def __init__(self, providers: List[Callable[[str], str]]):
        self.providers = providers

    def execute_prompt(self, prompt: str) → str:
        last_exception = None
        for provider in self.providers:
            try:
                # Attempt call with a strict 10-second timeout
                return provider(prompt)
            except Exception as e:
                print(f"[FALLBACK TRIGGERED] Provider {provider.__name__} failed: {e}. Ca")
                last_exception = e
                continue

        raise SystemError(f"All LLM providers in cascade failed! Final error: {last_exception}")

```

## 4. Runaway Recursive Loop Circuit Breaker

An autonomous agent with tools can easily enter an infinite loop:

- Agent runs `search("weather in London")`
- Tool returns error: `KeyError: temp`
- Agent repeats `search("weather in London")` forever, consuming \$500 in tokens in 20 minutes.

## The Strict Budget Token & Step Circuit Breaker

```
class AgentExecutionCircuitBreaker:
    def __init__(self, max_steps: int = 15, max_token_budget: int = 50_000, max_cost_dollars: float = 0.0):
        self.max_steps = max_steps
        self.max_token_budget = max_token_budget
        self.max_cost_dollars = max_cost_dollars

        self.current_steps = 0
        self.total_tokens_used = 0
        self.total_cost_accrued = 0.0

    def record_step(self, tokens_used: int, step_cost: float):
        self.current_steps += 1
        self.total_tokens_used += tokens_used
        self.total_cost_accrued += step_cost

        if self.current_steps > self.max_steps:
            raise TimeoutError(f"CIRCUIT BREAKER TRIPPED: Max step limit ({self.max_steps})")

        if self.total_tokens_used > self.max_token_budget:
            raise MemoryError(f"CIRCUIT BREAKER TRIPPED: Token budget ({self.max_token_budget})")

        if self.total_cost_accrued > self.max_cost_dollars:
            raise PermissionError(f"CIRCUIT BREAKER TRIPPED: Cost limit (${self.max_cost_dollars})")
```

Every production agent loop must wrap its execution inside this circuit breaker!